VerdeScan Technical Specification (Developer and AI-Agent Edition)

Oct 20, 2026 · @Samuel Mireku

1. How to use this spec

This document is the build contract for the VerdeScan PWA. It is written to be read by developers and by coding agents: every requirement is explicit, every interface has a schema, and every task has acceptance criteria. Where the spec and the product document disagree, this spec wins for implementation and the disagreement should be raised as an open question.

Conventions

- **MUST / SHOULD / MAY** follow RFC 2119 meaning. MUST items are release blockers.
- Identifiers in `code font` are literal: collection names, field names, route paths, env var names, error codes. Use them exactly.
- All TypeScript is `strict` mode. No `any`; use `unknown` and narrow.
- All timestamps are Firestore `Timestamp` in storage and ISO 8601 UTC strings in API payloads.
- All user-facing strings go through i18n keys. No hard-coded copy in components.
- Each backlog task in section 13 has an ID (`T-001`...). Commits and PRs reference the ID.
- If a requirement is ambiguous, do not guess silently: add it to section 14 and use the stated default.

Non-negotiable rules

1. AI model calls MUST go through the server-side diagnosis function except in the explicitly marked prototype path. API keys and prompts never ship in the client bundle.
2. Every Firebase AI Logic call MUST be protected by Firebase App Check. Enforcement for Firebase AI Logic becomes mandatory from 2 November 2026.
3. Model IDs MUST be read from Remote Config, never hard-coded. Google retires Gemini model versions on a regular schedule.
4. The app MUST remain usable offline for browsing guides and viewing history.

5. Pesticide and chemical advice MUST be organic-first and MUST NOT output dosing from model memory; chemical guidance only comes from reviewed knowledge-base entries that reference the product label.
6. User photos MUST have EXIF location data removed before upload.
7. Every diagnosis response MUST carry a confidence value and a `safetyNotes` array, even when empty.

Glossary

Term	Meaning
Diagnosis	One analysis request: images plus context in, ranked candidate causes plus treatment plan out
Candidate	One possible cause (disease, pest, deficiency, or stress) with a confidence score
Problem	A knowledge-base entry describing a disease, pest, deficiency, or stress
KB	The curated, expert-reviewed knowledge base in Firestore (`kb_*` collections)
Guide	Educational content: propagation, composting, recycled-plastic builds, and so on
Orchestrator	The Cloud Function that runs the diagnosis pipeline
Golden set	Expert-labelled test images used to measure diagnostic accuracy

2. System context and repository layout

The system is a Next.js PWA backed entirely by Firebase and Google Cloud. One monorepo holds the web app, Cloud Functions, shared types, and knowledge-base content so that types and schemas have a single source of truth.

Components

Component	Responsibility	Runs on
`apps/web`	PWA UI, camera capture, offline cache, i18n	Firebase App Hosting (Next.js)
`functions`	Diagnosis orchestrator, KB search, moderation, notifications	Cloud Functions 2nd gen (Node 20, TypeScript)
`packages/shared`	Zod schemas and TypeScript types shared by web and functions	Build-time only
`packages/kb`	Markdown/YAML knowledge-base source and seeding scripts	CI and local
`packages/eval`	Golden-set runner and accuracy reports	CI and local
Firebase services	Auth, Firestore, Storage, Remote Config, FCM, App Check, Analytics, Crashlytics	Google-managed
Gemini	Image and text reasoning via Firebase AI Logic or Vertex AI	Google-managed

Repository layout

```
verdescan/
  apps/
    web/
      src/
        app/                # Next.js App Router routes
        components/         # UI components
        features/           # diagnose, garden, guides, auth, settings
        lib/                # firebase client, i18n, offline db, analytics
        sw/                 # service worker source (Workbox)
      public/               # manifest, icons, static assets
  functions/
    src/
      diagnose/             # orchestrator, prompts, schema, safety
      kb/                   # retrieval and indexing
      moderation/
```

```
      notifications/
      index.ts                # function exports
  packages/
    shared/src/               # zod schemas, types, constants, error codes
    kb/
      content/                # problems/, crops/, guides/, treatments/ (md +
yaml)
      scripts/                # validate, translate, seed
    eval/
      golden/                 # labelled images + labels.json
      src/                    # runner, metrics
  firebase.json
  firestore.rules
  firestore.indexes.json
  storage.rules
  remoteconfig.template.json
  .github/workflows/          # ci.yml, deploy.yml, eval.yml
  AGENTS.md                   # instructions for coding agents (see below)
```

Environments

Env	Firebase project alias	Purpose	AI model source
`local`	emulators	Development with Auth, Firestore, Storage, Functions emulators	Mocked responses by default; real calls behind a flag
`dev`	`verdescan-dev`	Shared integration testing	Remote Config `dev` values, low quotas
`staging`	`verdescan-staging`	Pre-release and pilot rehearsal	Same models as prod
`prod`	`verdescan-prod`	Live users	Remote Config prod values

AGENTS.md for coding agents

Commit an `AGENTS.md` at the repo root containing: the non-negotiable rules from section 1, the repo layout above, the commands in section 12 (install, lint, typecheck, test, emulate, eval), the rule that schemas live in `packages/shared` and are imported, never redefined,

and the instruction to reference a backlog task ID in every change.

3. Stack and configuration

Versions are intentionally given as majors or ranges. At task `T-001`, pin exact versions in lockfiles and record them in `docs/versions.md`; do not upgrade majors mid-phase without a decision logged in section 14.

Technology choices

Area	Choice	Notes
Language	TypeScript (strict) everywhere	Shared types via `packages/shared`
Runtime	Node.js current LTS	Used by Functions and tooling
Web framework	Next.js (App Router), React	Server components for guide pages; client components for camera and offline
Styling	Tailwind CSS	Mobile-first; no runtime CSS-in-JS
State	React Query for server state, Zustand for UI state	No Redux
Forms and validation	React Hook Form with Zod	Zod schemas shared with backend
PWA	Workbox (service worker), Web App Manifest	See section 8
Local storage	IndexedDB via Dexie	Offline cache and upload queue
i18n	next-intl or equivalent ICU-message library	Language packs lazy-loaded
Backend	Cloud Functions 2nd gen, TypeScript	Callable functions plus HTTPS where needed
Database	Cloud Firestore (Native mode)	Offline persistence enabled on web
Files	Cloud Storage for Firebase	Signed, rules-scoped access

Area	Choice	Notes
Auth	Firebase Authentication	Google, phone OTP, email link
AI	Gemini via Firebase AI Logic SDK, Vertex AI backend on the server path	Model IDs from Remote Config
Retrieval	Firestore vector search over KB embeddings (default)	Alternative: Vertex AI Search; see section 14
Messaging	Firebase Cloud Messaging (web push)	Reminders and advisories
Observability	Analytics, Crashlytics, Performance Monitoring, Cloud Logging	Structured JSON logs
Testing	Vitest, React Testing Library, Playwright, Firebase Emulator Suite	See section 11
Quality	ESLint, Prettier, TypeScript project references	Enforced in CI
Package manager	pnpm workspaces	Single lockfile

Remote Config keys

Key	Type	Purpose
`ai_model_diagnosis`	string	Gemini model ID used by the orchestrator; choose the current stable Flash-class model from the Firebase AI Logic docs at implementation time
`ai_model_fallback`	string	Fallback model ID if the primary errors or is retired
`ai_max_output_tokens`	number	Output token cap per diagnosis
`ai_temperature`	number	Default 0.2 for diagnostic stability
`diag_free_monthly_limit`	number	Free-tier diagnoses per user per month

Key	Type	Purpose
`diag_image_max_px`	number	Longest image edge after compression, default 1024
`diag_min_confidence_answer`	number	Below this the app asks follow-ups instead of asserting a cause, default 0.55
`feature_outbreak_map`	boolean	Feature flag, off until Phase 3
`kb_version`	string	Active knowledge-base content version

Environment variables

Variable	Scope	Description
`NEXT_PUBLIC_FIREBASE_*`	web	Public Firebase web config (apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId)
`NEXT_PUBLIC_RECAPTCHA_SITE_KEY`	web	App Check provider site key
`NEXT_PUBLIC_DEFAULT_LOCALE`	web	Default `en`
`GCP_PROJECT`	functions	Provided by runtime
`VERTEX_LOCATION`	functions	Region for model calls
`KB_EMBEDDING_MODEL`	functions	Embedding model ID used for KB indexing and queries
`USE_AI_MOCK`	local only	`true` returns fixtures from `functions/src/diagnose/fixtures`

Secrets (service credentials, third-party keys) live in Secret Manager and are bound to functions at deploy time. Nothing secret is committed or placed in `NEXT_PUBLIC_*`.

Firebase project setup checklist

1. Create three Firebase projects (dev, staging, prod) and register a web app in each.
2. Enable Authentication providers: Google, Phone, Email link.
3. Create Firestore in Native mode in the chosen region; enable offline persistence in the client.

4. Create default Storage bucket; apply `storage.rules`.
5. Enable Firebase AI Logic with the Vertex AI Gemini API provider; link a Cloud Billing account (Blaze plan).
6. Register App Check with reCAPTCHA Enterprise for web; enforce for Firestore, Storage, Functions, and AI Logic.
7. Create Remote Config template from `remoteconfig.template.json`.
8. Set budget alerts and per-project quotas before any public traffic.

4. Data model

Firestore is the system of record. The types below live in `packages/shared/src/models.ts` as Zod schemas with inferred TypeScript types; the code here is the canonical shape. Client code reads and writes only the paths marked client-writable; everything else is written by Cloud Functions.

Collections

Path	Written by	Purpose
`users/{uid}`	client (own doc)	Profile and preferences
`users/{uid}/plants/{plantId}`	client	User's plants and beds
`users/{uid}/diagnoses/{diagnosisId}`	functions (create, update); client (feedback fields only)	Diagnosis requests and results
`users/{uid}/usage/{yyyymm}`	functions	Monthly diagnosis counters for limits
`kb_crops/{cropId}`	admin pipeline	Crop reference data
`kb_problems/{problemId}`	admin pipeline	Diseases, pests, deficiencies, stresses
`kb_treatments/{treatmentId}`	admin pipeline	Treatment and prevention methods

Path	Written by	Purpose
`kb_guides/{guideId}`	admin pipeline	Propagation, composting, recycled-material builds
`kb_chunks/{chunkId}`	admin pipeline	Text chunks plus embedding vectors for retrieval
`reports/{reportId}`	functions	Anonymised, location-fuzzed outbreak signals (Phase 3)
`expertReviews/{reviewId}`	functions, reviewer role	Flagged diagnoses and expert corrections

Canonical types

```
export type Locale = 'en' | 'tw' | 'fr' | 'ha' | 'sw' | 'es';
export type ProblemCategory = 'disease' | 'pest' | 'deficiency' | 'stress';
export type PlantPart = 'leaf' | 'stem' | 'fruit' | 'flower' | 'root' |
'whole';
export type DiagnosisStatus =
  | 'pending_upload' | 'processing' | 'needs_info' | 'complete' | 'failed' |
'flagged';

export interface UserProfile {
  uid: string;
  displayName?: string;
  locale: Locale;
  country?: string;            // ISO 3166-1 alpha-2
  regionGeohash?: string;      // precision 4 max (about 20 km); never exact
location
  crops: string[];             // kb_crops ids
  role: 'grower' | 'extension' | 'reviewer' | 'admin';
  plan: 'free' | 'plus' | 'org';
  trainingConsent: boolean;    // opt-in use of photos for model improvement
  createdAt: Timestamp;
}

export interface Plant {
  id: string;
```

```
  cropId: string;              // kb_crops id
  nickname?: string;
  plantedOn?: Timestamp;
  location: 'bed' | 'container' | 'field' | 'indoor' | 'greenhouse';
  sunExposure?: 'full' | 'partial' | 'shade';
  createdAt: Timestamp;
}

export interface DiagnosisImage {
  storagePath: string;         // users/{uid}/diagnoses/{id}/{n}.jpg
  part: PlantPart;
  widthPx: number;
  heightPx: number;
}

export interface DiagnosisContext {
  cropId?: string;             // may be unknown; plant ID can fill it
  plantId?: string;
  symptomsText?: string;       // max 500 chars
  daysSinceSymptoms?: number;
  recentWeather?: 'dry' | 'wet' | 'hot' | 'cold' | 'mixed';
  wateringFrequency?: 'daily' | 'weekly' | 'rare';
  recentFertilizer?: boolean;
  followUpAnswers?: Record<string, string>; // questionId -> answer
}

export interface Candidate {
  problemId: string;           // kb_problems id; MUST exist in KB
  category: ProblemCategory;
  confidence: number;          // 0..1
  evidence: string[];          // visible signs that support this candidate
  rulesOut: string[];          // signs that would argue against it
}

export interface Diagnosis {
  id: string;
  uid: string;
  status: DiagnosisStatus;
  images: DiagnosisImage[];
  context: DiagnosisContext;
  candidates: Candidate[];     // sorted by confidence desc; max 4
  topProblemId?: string;
  followUpQuestions: { id: string; text: string; options?: string[] }[];
```

```
  treatmentIds: string[];      // kb_treatments ids
  safetyNotes: string[];
  modelId: string;             // value used from Remote Config
  kbVersion: string;
  promptVersion: string;
  latencyMs?: number;
  userFeedback?: { helpful: boolean; confirmedProblemId?: string; outcome?:
'resolved' | 'improving' | 'no_change' | 'worse'; at: Timestamp };
  createdAt: Timestamp;
  updatedAt: Timestamp;
}

export interface KbProblem {
  id: string;                  // slug, e.g. tomato-early-blight
  category: ProblemCategory;
  cropIds: string[];           // crops it affects
  names: Record<Locale, string>;
  summary: Record<Locale, string>;
  symptoms: string[];          // plain-language visible signs
  lookAlikes: string[];        // problem ids easily confused with this one
  confirmSteps: string[];      // how a grower can verify at home
  treatmentIds: string[];
  preventionIds: string[];
  referenceImages: string[];   // storage paths, licence-cleared
  reviewedBy?: string;         // expert id
  reviewedAt?: Timestamp;
  version: number;
}

export interface KbTreatment {
  id: string;
  kind: 'cultural' | 'biological' | 'organic_input' | 'chemical';
  title: Record<Locale, string>;
  steps: Record<Locale, string[]>;
  materials: string[];
  costTier: 'free' | 'low' | 'medium' | 'high';
  safety: Record<Locale, string[]>;
  labelReferenceRequired: boolean; // true for chemical kind
  reviewedBy: string;
  version: number;
}

export interface KbGuide {
```

```
  id: string;
  topic: 'propagation' | 'seed' | 'scrap_regrow' | 'compost' | 'water' |
'recycled_plastic' | 'season_extension' | 'companion' | 'soil';
  title: Record<Locale, string>;
  body: Record<Locale, string>;   // markdown
  materials: string[];
  difficulty: 'easy' | 'medium' | 'hard';
  timeEstimate: string;
  safetyNotes: Record<Locale, string[]>; // e.g. which plastic types are safe
for food crops
  version: number;
}

export interface KbChunk {
  id: string;
  sourceType: 'problem' | 'treatment' | 'guide';
  sourceId: string;
  locale: Locale;
  text: string;
  embedding: number[];         // stored as Firestore vector
  cropIds: string[];
}
```

Required composite indexes

Collection	Fields	Used for
`users/{uid}/diagnoses`	`createdAt` desc	History list
`users/{uid}/diagnoses`	`status` asc, `createdAt` desc	Pending and flagged views
`kb_problems`	`cropIds` array-contains, `category` asc	Browse problems by crop
`kb_guides`	`topic` asc, `difficulty` asc	Guide browsing
`kb_chunks`	vector index on `embedding`, filter on `locale` and `cropIds`	Retrieval

Data rules

- `candidates[].problemId` MUST reference an existing `kb_problems` document; the orchestrator drops any model output that does not.
- Documents carry `version`; content edits increment it and the active `kb_version` in Remote Config selects the content set.
- Deleting a user MUST delete `users/{uid}` recursively and the matching Storage prefix.

5. Auth, security rules, and privacy

Security is deny-by-default: every rule below grants the minimum, and anything not listed is refused.

Authentication

- Providers: Google sign-in, phone OTP, and email link. Phone OTP is first-class because many target users do not use email.
- Anonymous sign-in MAY be used for the first diagnosis; the anonymous account MUST be linkable to a permanent credential without data loss.
- Roles (`grower`, `extension`, `reviewer`, `admin`) are stored as Firebase custom claims set only by an admin function. The `role` field on the user document mirrors the claim for display and MUST NOT be trusted for authorization.
- Sessions use Firebase ID tokens; Cloud Functions verify them automatically for callable functions.

App Check

- Provider: reCAPTCHA Enterprise on web.
- Enforcement MUST be turned on for Firestore, Cloud Storage, Cloud Functions, and Firebase AI Logic before launch. Firebase AI Logic enforcement is mandatory from 2 November 2026.
- Local development uses an App Check debug token stored per developer, never committed.

Firestore rules (`firestore.rules`)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() { return request.auth != null; }
    function isOwner(uid) { return signedIn() && request.auth.uid == uid; }
    function isReviewer() { return signedIn() && request.auth.token.role in
['reviewer', 'admin']; }

    match /users/{uid} {
      allow read: if isOwner(uid);
      allow create: if isOwner(uid)
        && request.resource.data.uid == uid
        && request.resource.data.role == 'grower'
        && request.resource.data.plan == 'free';
      allow update: if isOwner(uid)
        && request.resource.data.role == resource.data.role
        && request.resource.data.plan == resource.data.plan;
      allow delete: if false;   // deletion handled by deleteAccount function

      match /plants/{plantId} {
        allow read, write: if isOwner(uid);
      }
      match /diagnoses/{diagnosisId} {
        allow read: if isOwner(uid);
        allow update: if isOwner(uid)
          &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['userFeedba
ck']);
        allow create, delete: if false;   // created by createDiagnosis
function
      }
      match /usage/{month} {
        allow read: if isOwner(uid);
        allow write: if false;
      }
    }

    match /kb_crops/{id}      { allow read: if true; allow write: if false; }
    match /kb_problems/{id}   { allow read: if true; allow write: if false; }
    match /kb_treatments/{id} { allow read: if true; allow write: if false; }
    match /kb_guides/{id}     { allow read: if true; allow write: if false; }
```

```
    match /kb_chunks/{id}     { allow read, write: if false; }   // server
only
    match /reports/{id}       { allow read, write: if false; }   // server
only
    match /expertReviews/{id} { allow read, write: if isReviewer(); }
  }
}
```

Storage rules (`storage.rules`)

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /users/{uid}/diagnoses/{diagnosisId}/{file} {
      allow read: if request.auth != null && request.auth.uid == uid;
      allow write: if request.auth != null && request.auth.uid == uid
        && request.resource.size < 3 * 1024 * 1024
        && request.resource.contentType.matches('image/(jpeg|webp)');
      allow delete: if false;
    }
    match /{allPaths=**} { allow read, write: if false; }
  }
}
```

Privacy controls

Control	Requirement
EXIF stripping	Client re-encodes every image through a canvas before upload, which removes metadata; the server verifies no GPS tags remain and rejects the upload otherwise
Location	Only a coarse geohash (precision 4) is stored on the profile; outbreak reports use fuzzed cells
Training use	Photos are used for model improvement only when `trainingConsent` is true; default false
Consent	First-run consent screen records consent version and time; stored on the profile

Control	Requirement
Data export	`exportMyData` function returns a JSON archive of the user's documents and signed image URLs
Account deletion	`deleteAccount` function removes Auth user, Firestore subtree, and Storage prefix within 24 hours and logs the deletion
Logging	Logs MUST NOT contain image bytes, free-text symptoms, or precise location; log ids and counts only
Compliance baseline	GDPR principles and Ghana Data Protection Act 2012 (Act 843); confirm legal review before launch

Abuse and cost protection

- Per-user monthly diagnosis counters in `users/{uid}/usage/{yyyymm}` enforce `diag_free_monthly_limit` server-side.
- A per-user rate limit of 10 diagnosis requests per hour applies regardless of plan.
- Image count per diagnosis is capped at 4; image size at 3 MB after compression.
- Budget alerts trigger at 50%, 80%, and 100% of the monthly cloud budget; at 100% the orchestrator switches to the fallback model or returns a graceful limit message.

6. Backend contracts

All endpoints are Firebase callable functions unless noted. Request and response schemas are Zod objects in `packages/shared/src/api.ts`; both the web client and the functions import them. Callables MUST validate input with the schema and return typed errors.

Endpoint list

Function	Auth	Purpose	Idempotent
`createDiagnosis`	required	Reserve a diagnosis id, check limits, return signed upload paths	Yes, via `clientRequestId`

Function	Auth	Purpose	Idempotent
`submitDiagnosis`	required	Run the pipeline on uploaded images and return the result	Yes, returns stored result if already complete
`answerFollowUp`	required	Add follow-up answers and re-run the final reasoning step	No
`flagDiagnosis`	required	Send a diagnosis to the expert review queue	Yes
`searchGuides`	optional	Semantic and keyword search over KB guides and problems	Yes
`exportMyData`	required	Return a data export for the caller	Yes
`deleteAccount`	required	Delete the caller's account and data	Yes
`setUserRole`	admin only	Set custom claim role	Yes
`publishKbVersion`	admin only	Validate and activate a KB version	Yes

Schemas (`api.ts`)

```
import { z } from 'zod';

export const DiagnosisContextSchema = z.object({
  cropId: z.string().optional(),
  plantId: z.string().optional(),
  symptomsText: z.string().max(500).optional(),
  daysSinceSymptoms: z.number().int().min(0).max(365).optional(),
  recentWeather: z.enum(['dry', 'wet', 'hot', 'cold', 'mixed']).optional(),
  wateringFrequency: z.enum(['daily', 'weekly', 'rare']).optional(),
  recentFertilizer: z.boolean().optional(),
  followUpAnswers: z.record(z.string()).optional(),
});
```

```
export const CreateDiagnosisReq = z.object({
  clientRequestId: z.string().uuid(),
  imageParts: z.array(z.enum(['leaf', 'stem', 'fruit', 'flower', 'root',
'whole'])).min(1).max(4),
  context: DiagnosisContextSchema,
  locale: z.enum(['en', 'tw', 'fr', 'ha', 'sw', 'es']),
});
export const CreateDiagnosisRes = z.object({
  diagnosisId: z.string(),
  uploadPaths: z.array(z.string()).min(1).max(4), // storage paths the client
MUST upload to
  remainingThisMonth: z.number().int().nullable(), // null for unlimited
plans
});

export const SubmitDiagnosisReq = z.object({ diagnosisId: z.string() });

export const CandidateOut = z.object({
  problemId: z.string(),
  category: z.enum(['disease', 'pest', 'deficiency', 'stress']),
  confidence: z.number().min(0).max(1),
  evidence: z.array(z.string()),
  rulesOut: z.array(z.string()),
});

export const SubmitDiagnosisRes = z.object({
  diagnosisId: z.string(),
  status: z.enum(['complete', 'needs_info', 'failed', 'flagged']),
  candidates: z.array(CandidateOut).max(4),
  followUpQuestions: z.array(z.object({
    id: z.string(), text: z.string(), options:
z.array(z.string()).optional(),
  })),
  treatmentIds: z.array(z.string()),
  safetyNotes: z.array(z.string()),
  disclaimerKey: z.literal('diagnosis.firstLineGuidance'),
});

export const AnswerFollowUpReq = z.object({
  diagnosisId: z.string(),
  answers: z.record(z.string()).refine(a => Object.keys(a).length > 0),
});
```

```
export const SearchGuidesReq = z.object({
  query: z.string().min(2).max(200),
  locale: z.enum(['en', 'tw', 'fr', 'ha', 'sw', 'es']),
  cropId: z.string().optional(),
  limit: z.number().int().min(1).max(20).default(10),
});
```

Error codes

Functions throw `HttpsError` with a Firebase code plus `details.code` set to one of the app codes below. The client maps `details.code` to i18n keys of the form `errors.<CODE>`.

details.code	Firebase code	When	Client behaviour
`AUTH_REQUIRED`	`unauthenticated`	No valid token	Prompt sign-in
`LIMIT_REACHED`	`resource-exhausted`	Monthly or hourly limit exceeded	Show upgrade or wait message
`INVALID_INPUT`	`invalid-argument`	Schema validation failed	Show generic error; log id
`IMAGES_MISSING`	`failed-precondition`	Uploaded files not found at `uploadPaths`	Retry upload
`IMAGE_INVALID`	`invalid-argument`	Unsupported type, oversize, or corrupt	Ask for a new photo
`IMAGE_HAS_GPS`	`invalid-argument`	Location metadata present	Re-encode and re-upload automatically
`IMAGE_LOW_QUALITY`	`failed-precondition`	Blurry, dark, or no plant detected	Show framing tips
`MODEL_UNAVAILABLE`	`unavailable`	Primary and fallback model both failed	Queue and retry; show offline-style message
`KB_EMPTY`	`failed-precondition`	No KB entries for the crop and region	Return generic guidance and flag for content team

details.code	Firebase code	When	Client behaviour
INTERNAL	internal	Anything else	Show generic error; log id

Contract rules

- submitDiagnosis MUST complete within 60 seconds (function timeout) and SHOULD complete in under 10 seconds at the 50th percentile on 4G.
- The function MUST write the final Diagnosis document before returning; the client treats the Firestore document as the source of truth and MAY also subscribe to it.
- If a request with the same clientRequestId arrives twice, the second returns the first result and MUST NOT consume quota or call the model again.
- Responses MUST NOT include raw model text; only validated, schema-conforming fields.

7. AI diagnosis pipeline

Design principle: the model classifies against a closed set of known problems retrieved from the curated KB, rather than answering from open-ended memory. This keeps answers reviewable, lets treatments come from vetted content, and makes accuracy measurable against the golden set.

Code lives in functions/src/diagnose/. Pipeline mode is controlled by Remote Config key diag_pipeline_mode (two_stage default, single for cost-reduced mode); add this key to remoteconfig.template.json.

Pipeline steps (executed by submitDiagnosis)

1. **Load and validate.** Fetch the diagnosis document and images from Storage. Reject if files are missing, over 3 MB, not JPEG or WebP, or contain GPS metadata (IMAGES_MISSING, IMAGE_INVALID, IMAGE_HAS_GPS). Set status processing.
2. **Observe (stage 1, vision).** Call the model with the images and the observation prompt. Output: structured observations (plant part, crop guess, visible symptom descriptors, presence of insects or eggs, image quality). If imageQuality.usable is false, return IMAGE_LOW_QUALITY.

3. **Retrieve.** Build a query from observations plus user context text; embed it with `KB_EMBEDDING_MODEL`; run a vector search on `kb_chunks`, filtered by `cropIds` when the crop is known (user-supplied crop wins over the guess). Collect the top 12 distinct `problemId` values. If none, throw `KB_EMPTY`.
4. **Reason (stage 2, vision plus text).** Call the model with the images, observations, context, and the shortlist of problem summaries. The model ranks up to 4 candidates strictly from the shortlist, or returns an empty list.
5. **Validate.** Parse against `ModelOutputSchema`. Drop candidates whose `problemId` is not in the shortlist. Re-sort by confidence and cap at 4.
6. **Decide.** Apply the thresholds below to choose `complete`, `needs_info`, or `flagged`.
7. **Attach content.** Look up `treatmentIds`, prevention items, and safety notes from the KB entries of the top candidates. No treatment text comes from the model.
8. **Persist and respond.** Write the `Diagnosis` document (including `modelId`, `kbVersion`, `promptVersion`, `latencyMs`), increment the usage counter, and return `SubmitDiagnosisRes`.

In `single` mode, steps 2 to 4 merge into one call that receives images and the retrieved shortlist built from user context only.

Decision thresholds (provisional)

Thresholds read from Remote Config and MUST be recalibrated against the golden set before launch; model-reported confidence is not a calibrated probability until a mapping is fitted (section 11).

Condition	Outcome
Candidate list empty	`needs_info`; ask for more photos and a different plant part
Top confidence below `diag_min_confidence_answer` (0.55)	`needs_info`; return follow-up questions built from `lookAlikes` and `confirmSteps`
Top two candidates within 0.10 of each other	`needs_info`; ask the single question that best separates them
Top candidate category is `disease` or `pest` with high spread risk and confidence below 0.75	`complete`, with a safety note advising local expert confirmation

Condition	Outcome
Model output fails schema twice	`flagged`; send to expert review queue
Otherwise	`complete`

Model output schema (`ModelOutputSchema`)

```
export const ObservationSchema = z.object({
  imageQuality: z.object({ usable: z.boolean(), issues:
z.array(z.enum(['blurry', 'dark', 'overexposed', 'no_plant', 'too_far',
'other'])) }),
  plantPart: z.enum(['leaf', 'stem', 'fruit', 'flower', 'root', 'whole',
'unknown']),
  cropGuess: z.object({ cropId: z.string().nullable(), confidence:
z.number().min(0).max(1) }),
  symptoms: z.array(z.object({
    descriptor: z.string().max(120),     // e.g. brown concentric rings on
lower leaves
    location: z.string().max(60),
    extent: z.enum(['isolated', 'scattered', 'widespread']),
  })).max(8),
  organismsVisible: z.array(z.object({ kind: z.enum(['insect', 'egg',
'larva', 'mite', 'web', 'fungal_growth', 'other']), description:
z.string().max(100) })).max(5),
});

export const ModelOutputSchema = z.object({
  candidates: z.array(z.object({
    problemId: z.string(),
    category: z.enum(['disease', 'pest', 'deficiency', 'stress']),
    confidence: z.number().min(0).max(1),
    evidence: z.array(z.string().max(160)).min(1).max(4),
    rulesOut: z.array(z.string().max(160)).max(3),
  })).max(4),
  followUpQuestions: z.array(z.object({ id: z.string(), text:
z.string().max(140), options: z.array(z.string().max(40)).max(5).optional()
})).max(3),
});
```

Call the model with structured output enabled (JSON response MIME type and a response schema derived from these Zod schemas) so responses are machine-parseable.

Prompt templates

Prompts are stored as versioned TypeScript constants in `functions/src/diagnose/prompts/`. `promptVersion` strings follow `diag-v<N>`; any prompt change increments N and MUST pass the golden-set evaluation before deployment.

Stage 1 (observe) system prompt:

```
You are a plant-health observation assistant. Describe only what is visibly
present in the supplied photos.
Do not name any disease, pest, or deficiency in this step.
Report plant part, likely crop if clear, symptom descriptors with location
and extent, and any visible organisms or eggs.
Report image quality honestly. If the photo is blurry, dark, shows no plant,
or shows a person or animal, mark it not usable.
Output JSON that matches the provided schema and nothing else.
```

Stage 2 (reason) system prompt:

```
You are an agronomy decision-support assistant for small-scale growers.
You receive photos, structured observations, grower context, and a shortlist
of known problems with summaries.
Rank up to four candidates strictly from the shortlist using the problemId
values given. Never invent a problemId.
For each candidate list the visible evidence that supports it and the signs
that would argue against it.
Use a lower confidence when symptoms overlap with other shortlist items or
when photos are limited.
If nothing in the shortlist fits, return an empty candidates list and ask up
to three short follow-up questions.
Do not give treatment, dosage, or product advice. Treatment is attached
separately from reviewed content.
Output JSON that matches the provided schema and nothing else.
```

User content for stage 2 is assembled in this order: images, observation JSON, context JSON, shortlist JSON (`problemId`, `category`, `names.en`, `summary.en`, `symptoms`, `lookAlikes`). All shortlist text is sent in English; localisation happens when results are rendered from the KB.

Safety rules (enforced in code, not only in prompts)

- Candidates must be KB-backed; non-matching ids are dropped.

- Chemical treatments are attached only if the KB entry has `labelReferenceRequired: true` and the result includes the safety note key `safety.followProductLabel`.
- Photos showing people, faces, or animals are rejected as `IMAGE_INVALID`; no analysis is attempted.
- Free-text user symptoms are passed to the model inside a delimited block and treated as data; instructions inside that text MUST be ignored (prompt-injection defence). Truncate to 500 characters and strip control characters.
- Every response includes the disclaimer key `diagnosis.firstLineGuidance` telling users this is first-line guidance and to consult a local expert for high-stakes decisions.

Reliability and cost

- Retry the primary model once on transient errors, then switch to `ai_model_fallback`; if both fail, throw `MODEL_UNAVAILABLE`.
- Cache results by SHA-256 of the compressed image bytes plus `kbVersion` plus `promptVersion` for 7 days to avoid paying twice for identical photos; cache hits skip the model call but still count toward usage only if the user did not previously receive that result.
- Record tokens in, tokens out, latency per stage, and model id in structured logs for cost tracking.
- Target cost per diagnosis is tracked in the monitoring dashboard (section 12); a regression of more than 20% week over week raises an alert.

8. PWA and offline specification

Goal: a grower with a low-end Android phone and weak connectivity can install the app from a link, browse every cached guide, review past diagnoses, take photos offline, and have them analysed automatically when signal returns.

Web App Manifest (`apps/web/public/manifest.webmanifest`)

Field	Value
`name` / `short_name`	VerdeScan / VerdeScan
`start_url`	`/?source=pwa`
`display`	`standalone`

Field	Value
`orientation`	`portrait`
`theme_color` / `background_color`	Set from brand tokens (green primary, off-white background)
`icons`	192, 512, and maskable 512 PNG
`shortcuts`	New diagnosis (`/diagnose/new`), My Garden (`/garden`), Guides (`/guides`)
`lang`	Follows user locale where supported

Service worker caching strategies (Workbox)

Resource	Strategy	Details
App shell (HTML routes, JS, CSS, fonts)	Precache with revisioning	Updated on new deploy; prompt user to refresh when a new version is waiting
KB guides and problem pages	Stale-while-revalidate	Cache name `kb-v{kb_version}` ; max 400 entries, 30 days
KB images (reference photos)	Cache-first	Cache name `kb-img` ; max 300 entries, 60 days
Region starter pack (top crops and problems for user's region)	Precached on first run when online	Fetched as one bundle from `/api/kb-pack?region=…&locale=…` ; target under 5 MB
Firestore and Functions API calls	Network-only (SDK handles its own offline persistence)	Do not cache authenticated API responses in the service worker
User diagnosis photos	Not cached by the service worker	Thumbnails kept in IndexedDB only
Navigation fallback	Offline page	Shows cached guides and queued items when a route is unavailable

IndexedDB stores (Dexie)

```
export class LocalDB extends Dexie {
  queue!: Table<QueuedDiagnosis, string>;
  history!: Table<CachedDiagnosis, string>;
  bookmarks!: Table<Bookmark, string>;
  meta!: Table<{ key: string; value: unknown }, string>;
  constructor() {
    super('verdescan');
    this.version(1).stores({
      queue: 'id, state, createdAt',
      history: 'id, createdAt, topProblemId',
      bookmarks: 'id, kind, createdAt',
      meta: 'key',
    });
  }
}

export interface QueuedDiagnosis {
  id: string;                    // equals clientRequestId (uuid)
  state: 'queued' | 'creating' | 'uploading' | 'submitting' | 'done' |
'error';
  images: { part: PlantPart; blob: Blob; widthPx: number; heightPx: number }
[];
  context: DiagnosisContext;
  locale: Locale;
  diagnosisId?: string;          // set after createDiagnosis succeeds
  attempts: number;
  lastError?: string;
  createdAt: number;             // epoch ms
}
```

Offline upload queue

1. Capture produces compressed, EXIF-stripped blobs; the item is stored with state `queued`.
2. When online and the app is in the foreground, a queue runner processes one item at a time: `createDiagnosis` (state `creating`), upload each blob to `uploadPaths` (state `uploading`), `submitDiagnosis` (state `submitting`), then store the result in `history` and set state `done`.

3. Each step is idempotent; on restart the runner resumes from the item's stored state using `clientRequestId` and `diagnosisId`.
4. Failures increment `attempts` and use exponential backoff (30 s, 2 min, 10 min, 1 hour); after 5 attempts the item becomes `error` and the user can retry or delete it.
5. Triggers: `online` event, app foreground (`visibilitychange`), app start, and Background Sync where supported. Background Sync is not available on all browsers (notably iOS Safari), so foreground triggers are the baseline and MUST work alone.
6. While an item is `queued`, the UI shows a pending card with the photo thumbnail and the offline checklist of likely causes from the cached KB for the chosen crop.
7. Blob storage is bounded: maximum 20 queued items or 50 MB; beyond that, capture is blocked with a clear message.

Offline behaviours by feature

Feature	Offline behaviour
Browse guides and problems	Full, from cache (region pack plus visited pages)
New diagnosis	Capture and queue; offline checklist shown; result arrives when online
View history	From Firestore persistent cache and `history` store
Edit My Garden	Writes queue in Firestore's offline cache and sync automatically
Search guides	Keyword search over cached titles only; semantic search needs a connection
Sign in	Requires a connection the first time; existing sessions persist

Install and update

- Show an install prompt after the user's second completed diagnosis or first saved guide, using `beforeinstallprompt` where available and instructions for Safari's Add to Home Screen otherwise.
- When a new service worker is waiting, show a non-blocking refresh banner; never force-reload during a capture or an upload.

9. Frontend specification

Routes (Next.js App Router, `apps/web/src/app`)

Route	Rendering	Auth	Purpose
`/`	Server	none	Landing, install prompt, language picker
`/diagnose/new`	Client	optional (anonymous allowed)	Camera capture, context questions
`/diagnose/[id]`	Client	owner	Result, follow-ups, treatment plan, feedback
`/garden`	Client	required	My Garden: plants, history, reminders
`/garden/plants/[id]`	Client	owner	Plant detail and its diagnoses
`/guides`	Server	none	Guide index with topic and crop filters
`/guides/[topic]/[id]`	Server, cached	none	Guide page, public and indexable for SEO
`/problems/[id]`	Server, cached	none	Problem reference page (symptoms, look-alikes, prevention)
`/search`	Client	none	Semantic and keyword search
`/settings`	Client	required	Language, region, consent, data export, delete account
`/offline`	Static	none	Offline fallback

Feature modules (`apps/web/src/features`)

Module	Key components	Notes
`diagnose`	`CameraCapture`, `FramingGuide`, `ContextForm`, `ResultCard`, `CandidateList`, `FollowUpPrompt`, `TreatmentPlan`, `FeedbackBar`	Wraps queue runner; shows pending state for queued items
`garden`	`PlantList`, `PlantForm`, `DiagnosisTimeline`, `ReminderList`	Reads Firestore with offline persistence
`guides`	`GuideCard`, `GuideBody`, `StepList`, `MaterialsList`, `SafetyCallout`, `BookmarkButton`	Markdown renderer with sanitised output
`auth`	`SignInSheet`, `PhoneOtpForm`, `LinkAnonymous`	Anonymous to permanent linking flow
`settings`	`LanguagePicker`, `ConsentPanel`, `DataExport`, `DeleteAccount`	Calls `exportMyData` and `deleteAccount`
`shared`	`OfflineBanner`, `UpdateBanner`, `ErrorBoundary`, `VoiceReadAloud`	Read-aloud uses the Web Speech API where available

Image capture and compression

- Use `getUserMedia` with the rear camera preferred; fall back to a file input with `capture` for browsers where live preview fails.
- `FramingGuide` overlays tips: one close-up of the affected area, one of the whole plant, daylight, no flash, avoid fingers over the symptom.
- Compression pipeline (in `lib/image.ts`): decode, resize so the longest edge is `diag_image_max_px` (default 1024), draw to canvas, re-encode as JPEG at quality 0.8 (WebP when supported and smaller). Drawing through a canvas strips EXIF metadata.
- Run a quick on-device quality check (Laplacian variance for blur and mean luminance for darkness); warn the user and offer retake, but do not block submission.
- Maximum 4 images per diagnosis; show thumbnails with a part label selector.

State management

- Server state (Firestore reads, callable results): React Query with Firestore listeners for live diagnosis status.
- UI state (capture wizard step, sheet visibility): Zustand stores per feature.
- Persistent local state: Dexie only (section 8); no `localStorage` for anything beyond a locale hint.
- Forms: React Hook Form with Zod resolvers using shared schemas.

Internationalisation

- Initial locales: `en`, `tw`, `fr`, `ha`, `sw`, `es`. `en` is the source; others fall back to `en` per missing key.
- Message files in `apps/web/src/messages/<locale>.json`; keys are dot-namespaced (`diagnosis.firstLineGuidance`, `errors.LIMIT_REACHED`).
- KB content is localised in Firestore fields typed `Record<Locale, string>`; the UI picks the user locale and falls back to `en`.
- Safety text (treatment safety, disclaimers) in any new language MUST be reviewed by a human before release; machine translation alone is not sufficient for these fields.
- Support right-to-left layout readiness by using logical CSS properties.

Accessibility and low-literacy support

- Target WCAG 2.2 AA: contrast, focus order, labels, touch targets of at least 44 px.
- Every diagnosis result and guide step has a read-aloud control.
- Icons accompany all primary actions; avoid text-only critical controls.
- Respect reduced-motion and system font-size settings.

Performance budgets (CI-enforced with Lighthouse CI)

Metric	Budget	Context
Initial JS on `/diagnose/new`	180 KB gzipped or less	Mid-range Android on slow 4G profile
Largest Contentful Paint	2.5 s or less	Guide pages and landing
Interaction to Next Paint	200 ms or less	Capture and form flows

Metric	Budget	Context
Cumulative Layout Shift	0.1 or less	All routes
Compressed upload size per diagnosis	1.5 MB or less for 3 photos	Data cost control
PWA installability check	Pass	Lighthouse PWA audit

Design tokens

Define colour, spacing, radius, and type scale as CSS variables in `apps/web/src/styles/tokens.css` and map them in `tailwind.config`. Tokens are the only place brand values live. Provide light and dark themes from day one.

10. Knowledge base

The KB is the product's accuracy and safety backbone. Source content lives in Git as Markdown with YAML frontmatter under `packages/kb/content`, is validated in CI, reviewed by an agronomist, and then seeded to Firestore by script. Git history is the audit trail for every content change.

Directory layout

```
packages/kb/content/
  crops/            tomato.yaml, pepper.yaml, ...
  problems/         tomato-early-blight.md, aphids-general.md, nitrogen-
deficiency.md, ...
  treatments/       neem-oil-spray.md, crop-rotation.md, ...
  guides/           propagation/, seed/, scrap-regrow/, compost/, water/,
recycled-plastic/, ...
  i18n/             translations keyed by id and locale
```

Problem file format

Each problem is one Markdown file. Frontmatter holds structured fields mapped to `KbProblem`; the body holds long-form sections used for retrieval chunks.

```
---
id: tomato-early-blight
category: disease
cropIds: [tomato, potato]
names:
  en: Early blight
symptoms:
  - Brown spots with concentric rings on older lower leaves
  - Yellow halo around spots
  - Spots can spread to stems and fruit
lookAlikes: [tomato-septoria-leaf-spot, tomato-late-blight]
confirmSteps:
  - Check whether spots start on the oldest leaves
  - Look for target-like rings when the leaf is held to the light
treatmentIds: [remove-infected-leaves, copper-based-fungicide-label, mulch-
and-stake]
preventionIds: [crop-rotation, avoid-overhead-watering, spacing]
reviewedBy: expert-001
reviewedAt: 2026-11-01
version: 1
---

## Summary
One short paragraph in plain language.

## What it looks like
Longer description of symptoms and how they progress.

## What to do now
Steps written for a grower with limited tools.

## How to prevent it
Cultural practices that reduce risk next season.
```

Validation rules (run by `pnpm kb:validate`, required in CI)

- Frontmatter parses and matches the Zod schema for its type.
- Every id referenced (`cropIds`, `lookAlikes`, `treatmentIds`, `preventionIds`) exists.
- Every `chemical` treatment has `labelReferenceRequired: true` and a safety block.
- Every problem has at least 3 symptoms, 1 confirm step, and 1 treatment.
- `reviewedBy` and `reviewedAt` are present before an entry can be marked publishable.

- No dosage numbers appear in any treatment unless the treatment is marked label-referenced and cites the product label.
- Reference images carry a licence field and source URL; only licence-cleared images are published.
- Safety fields exist for every locale that has translated body text.

Seeding and indexing

1. `pnpm kb:seed --env <env>` writes validated content to the `kb_*` collections with the `version` increment.
2. The indexer splits each problem, treatment, and guide into chunks by section (about 150 to 300 words each), embeds each chunk with `KB_EMBEDDING_MODEL`, and writes `kb_chunks` documents with `cropIds` and `locale`.
3. `publishKbVersion` runs integrity checks (no dangling ids, every chunk has an embedding), then updates Remote Config `kb_version`.
4. Clients fetch the region starter pack using the active `kb_version`, so cached content updates when the version changes.

Review workflow

1. Author or AI-assisted draft opens a pull request with content changes. AI-drafted content MUST be labelled in the PR and is never published without human review.
2. CI runs validation and the golden-set evaluation if problem lists changed.
3. An agronomist reviewer approves in the PR and sets `reviewedBy` and `reviewedAt`.
4. Merge to `main` deploys content to `dev`; promotion to `staging` and `prod` follows the release process in section 12.
5. Expert corrections from `expertReviews` and user feedback become issues labelled `kb-correction`.

MVP content targets (proposed; adjust with the agronomy partner)

Item	Target
Crops	10 priority crops for the pilot region, for example tomato, pepper, okra, maize, cassava, cabbage, onion, garden egg, plantain, leafy greens
Problems per crop	6 to 10 across disease, pest, deficiency, and stress

Item	Target
Cross-crop problems	General pests, common deficiencies, light, and water stress
Treatments	Organic and cultural first; chemical entries only where reviewed and label-referenced
Guides	Seed starting, cuttings and division, 12 scrap-regrow guides, composting, worm bin, rainwater capture, 6 recycled-plastic builds (bottle planter, self-watering container, vertical tower, mini tunnel, drip irrigation, seed tray)
Locales at launch	`en` complete; `tw` and `fr` for safety-critical content

Recycled-plastic guides MUST include a safety callout on plastic types and food-contact suitability (for example, avoid growing edible crops in containers that held chemicals), reviewed by a qualified person before publication.

11. Testing and evaluation

Test layers

Layer	Tooling	Scope	Runs
Unit	Vitest	Pure functions: image compression helpers, threshold logic, schema validation, prompt assembly, error mapping	Every commit
Component	React Testing Library	Capture wizard, result card, follow-up flow, offline banner	Every commit
Rules tests	Firebase Emulator Suite with the rules unit testing library	Every allow and deny case in `firestore.rules` and `storage.rules`	Every commit touching rules

Layer	Tooling	Scope	Runs
Function integration	Vitest against emulators with `USE_AI_MOCK=true`	`createDiagnosis`, `submitDiagnosis`, idempotency, limits, error codes	Every pull request
End to end	Playwright	Install, offline capture then online sync, sign-in, diagnosis, guide browsing	Every pull request to `main`; nightly on staging
Performance	Lighthouse CI	Budgets in section 9	Every pull request to `main`
Accessibility	axe in Playwright	Key flows must have no serious violations	Every pull request to `main`
Model evaluation	`packages/eval` runner	Golden set accuracy and safety checks	On prompt, model, KB-schema, or retrieval changes; weekly

Required unit and integration cases

- Duplicate `clientRequestId` returns the same result and does not call the model or consume quota.
- A model returning an unknown `problemId` has that candidate dropped.
- Free-text symptoms containing instruction-like text do not change pipeline behaviour (prompt-injection test with fixture strings).
- Images with GPS metadata are rejected with `IMAGE_HAS_GPS`.
- Primary model failure triggers one retry then the fallback model; both failing yields `MODEL_UNAVAILABLE`.
- Thresholds produce the outcomes in the section 7 decision table for each boundary value.
- Offline queue resumes correctly after reload at each state of `QueuedDiagnosis`.
- Rules: a user cannot read another user's diagnoses, cannot change `role` or `plan`, and cannot create diagnosis documents directly.

Golden set

The golden set is a collection of expert-labelled photos that measures real diagnostic quality. It lives in `packages/eval/golden` (images in Cloud Storage or Git LFS, labels in Git).

```
export interface GoldenCase {
  id: string;
  imagePaths: string[];           // 1 to 4 images
  parts: PlantPart[];
  cropId: string;
  context?: DiagnosisContext;
  expectedProblemIds: string[];   // acceptable answers, first is primary
  category: ProblemCategory | 'healthy' | 'unknown';
  difficulty: 'clear' | 'ambiguous';
  region: string;                 // ISO country or geohash prefix
  labelledBy: string;             // expert id
  licence: string;                // image rights
}
```

Requirements:

- Initial target: at least 30 cases per priority crop pair of problem types, growing each phase; include healthy plants and out-of-scope photos (people, pets, non-plants) as negative cases.
- Split into `dev` (used while tuning prompts) and `holdout` (used only for release gates; never used for tuning).
- Real photos from the pilot region MUST be a large share; stock images alone overstate accuracy.
- Labels are made by an agronomist; disputed cases are marked `ambiguous` and excluded from the headline accuracy number but still reported.

Metrics (reported by `pnpm eval`)

Metric	Definition
Top-1 accuracy	Primary candidate matches any `expectedProblemIds`
Top-3 accuracy	Any of the top three candidates matches

Metric	Definition
Per-category recall	Recall for disease, pest, deficiency, and stress separately
Abstain rate	Share of cases returned as `needs_info`
Wrong-confident rate	Share of cases where status is `complete`, confidence is high, and the answer is wrong; this is the key safety metric
Negative-case pass rate	Out-of-scope photos correctly rejected
Calibration error	Expected calibration error between reported confidence and observed accuracy
p50 and p95 latency	Per stage and end to end
Cost per diagnosis	Tokens times current price per model

Release gates

Gate values are set jointly by the founder and the agronomy partner before the Phase 1 gate, and recorded in `packages/eval/gates.ts`. The CI job fails a pull request if holdout top-3 accuracy drops, wrong-confident rate rises, or negative-case pass rate falls compared with the main branch baseline.

Calibration

Model-reported confidence is treated as an ordinal score until calibrated. After the first full golden-set run, fit a monotonic mapping (for example isotonic regression) from reported confidence to observed accuracy, store it in `functions/src/diagnose/calibration.json`, apply it in step 6 of the pipeline, and then re-tune the thresholds in section 7. Re-fit after every model or prompt change.

12. DevOps

Standard commands

These scripts are defined in the root `package.json` and are the only commands a developer or coding agent needs. Keep this list in sync with `AGENTS.md`.

```
pnpm install                  # install all workspaces
pnpm dev                      # web app with Firebase emulators and AI mock
pnpm emulators                # Auth, Firestore, Storage, Functions emulators
only
pnpm lint                     # ESLint across workspaces
pnpm typecheck                # tsc with project references
pnpm test                     # unit and component tests
pnpm test:rules               # Firestore and Storage rules tests on emulators
pnpm test:functions           # function integration tests on emulators
pnpm test:e2e                 # Playwright
pnpm kb:validate              # validate knowledge-base content
pnpm kb:seed --env <env>      # seed KB to Firestore
pnpm eval                     # run golden-set evaluation (needs credentials)
pnpm build                    # build web app and functions
pnpm deploy:<env>             # deploy to dev, staging, or prod (CI only for
staging and prod)
```

Branching and releases

- Trunk-based: short-lived feature branches off `main`, squash-merged by pull request.
- Pull requests need passing CI and one review; changes to `functions/src/diagnose`, rules files, or `packages/kb` also need review from the designated owner.
- Merge to `main` auto-deploys to `dev`. Tag `v*-rc` promotes to `staging`. Tag `v*` promotes to `prod` after manual approval.
- Database and KB changes are backward compatible for one release so clients with cached bundles keep working.

CI workflows (`.github/workflows`)

Workflow	Trigger	Steps
`ci.yml`	Pull request	Install, lint, typecheck, unit and component tests, rules tests, function tests, KB validation, build
`e2e.yml`	Pull request to `main`, nightly	Playwright against emulators; axe checks; Lighthouse CI budgets

Workflow	Trigger	Steps
`eval.yml`	Changes to prompts, retrieval, KB schema; weekly	Run golden-set evaluation; compare to baseline; post report as a PR comment
`deploy.yml`	Merge to `main`, tags	Authenticate with workload identity (no long-lived keys), deploy hosting, functions, rules, indexes, Remote Config, and KB seed for the target environment

Secrets and access

- CI authenticates to Google Cloud through workload identity federation; no service account key files in the repo or in CI secrets.
- Runtime secrets live in Secret Manager; functions declare which secrets they bind.
- Production access uses least-privilege roles; only CI deploys to staging and prod.

Monitoring and alerts

Signal	Source	Alert when
Diagnosis latency p95	Cloud Logging metrics	Above 15 s for 15 minutes
Diagnosis error rate	Function logs by `details.code`	Above 3% for 15 minutes
Model fallback rate	Logs	Above 10% for 30 minutes
Cost per diagnosis	Logs and billing export	Up more than 20% week over week
Daily AI spend	Billing budget	50%, 80%, 100% of monthly budget
Wrong-confident rate	Weekly eval and user feedback	Above the release-gate baseline
Client errors	Crashlytics	New crash cluster affecting more than 1% of sessions
Core Web Vitals	Performance Monitoring	LCP or INP regression beyond budget

Signal	Source	Alert when
App Check rejections	Firebase console metrics	Sudden spike (possible abuse)

Structured log format

Functions emit one JSON line per pipeline stage with these fields: `severity`, `message`, `diagnosisId`, `uidHash` (salted hash, never the raw uid), `stage`, `modelId`, `promptVersion`, `kbVersion`, `tokensIn`, `tokensOut`, `latencyMs`, `errorCode`. Image bytes, free text, and precise locations MUST NOT be logged.

Backups and recovery

- Enable scheduled Firestore backups (daily, 14-day retention) and point-in-time recovery on prod.
- Storage has object versioning disabled for user photos (privacy) but enabled for KB assets.
- Document a restore drill and run it once before the pilot starts.

13. Phase 1 implementation backlog

This is the ordered work for the MVP (roadmap Phase 1, preceded by Phase 0 content and setup). Tasks with no unmet dependency can run in parallel. Reference the task ID in every branch name, commit, and pull request.

Definition of done (applies to every task)

- Code is typed, linted, and covered by tests appropriate to its layer (section 11).
- No new `any`, no hard-coded user-facing strings, no secrets in the repo.
- Schemas are imported from `packages/shared`, never redefined.
- Docs updated if an interface, env var, or command changed.
- CI is green and the acceptance criteria below are demonstrably met.

Backlog

ID	Task	Depends on	Acceptance criteria
`T-001`	Bootstrap monorepo: pnpm workspaces, strict TypeScript, ESLint, Prettier, CI skeleton, `AGENTS.md`, `docs/versions.md`	none	`pnpm lint`, `typecheck`, `test` pass on a clean checkout; CI runs green on a pull request
`T-002`	Create dev, staging, and prod Firebase projects; `firebase.json`; local emulators	none	`pnpm emulators` starts Auth, Firestore, Storage, Functions; project aliases configured
`T-003`	Shared package: Zod schemas and types for models (section 4) and API (section 6), error codes	`T-001`	Both web and functions import the schemas; schema unit tests cover valid and invalid samples
`T-004`	Firestore and Storage rules plus rules tests	`T-002`, `T-003`	Every allow and deny case from section 5 and section 11 passes on emulators
`T-005`	App Check on web and functions, debug-token workflow	`T-002`	Unattested requests are rejected in staging; local debug token works and is not committed
`T-006`	Auth: Google, phone OTP, anonymous with account linking	`T-002`	E2E sign-in for each provider on emulators; linking an anonymous account preserves its data
`T-007`	Web shell: Next.js App Router, design tokens, layout, i18n scaffold, offline banner	`T-001`	Lighthouse baseline recorded; lint rule blocks string literals in JSX; locale switch works
`T-008`	PWA baseline: manifest, Workbox precache, offline page, install prompt logic	`T-007`	Lighthouse PWA audit passes; offline page appears when network is off

ID	Task	Depends on	Acceptance criteria
`T-009`	Image capture, compression, EXIF-safe re-encode, quality check	`T-007`	Output longest edge at most 1024 px; fixture with GPS EXIF has none after processing; blur and dark warnings appear on test images
`T-010`	Dexie stores and offline queue runner with stub backend	`T-009`	State-machine tests for each transition; queue resumes after reload at every state; backoff schedule verified
`T-011`	`createDiagnosis`: limits, signed upload paths, idempotency	`T-003`, `T-004`	Duplicate `clientRequestId` returns same response without consuming quota; limit errors return `LIMIT_REACHED`
`T-012`	KB schemas, content validator, seed script, sample content for 3 crops	`T-003`	`pnpm kb:validate` passes and fails correctly on seeded bad fixtures; `pnpm kb:seed` populates emulator
`T-013`	KB indexer and retrieval function (embeddings, vector search, crop filter)	`T-012`	Fixture queries return expected `problemId` values in the top 12; crop filter excludes other crops
`T-014`	Orchestrator stages 1 and 2 with AI mock, real model behind flag, Remote Config model IDs, fallback	`T-011`, `T-013`	Pipeline tests pass with mock; real-model smoke test passes on dev; fallback triggers on forced failure
`T-015`	Decision logic, safety rules, KB content attachment	`T-014`	Every row of the section 7 threshold table has a passing test; unknown `problemId` dropped; no treatment text from the model

ID	Task	Depends on	Acceptance criteria
`T-016`	`submitDiagnosis`, `answerFollowUp`, `flagDiagnosis`	`T-015`	Integration tests cover success, `needs_info`, flagged, and each error code; Diagnosis document written before response
`T-017`	Diagnose UI: wizard, result, follow-ups, treatment plan, feedback	`T-010`, `T-016`	E2E photo to result with mock AI; axe reports no serious violations; queued items show pending card
`T-018`	Region starter pack endpoint and offline guide caching	`T-008`, `T-012`	Guides open with network off after first run; pack is 5 MB or less
`T-019`	Guide and problem pages (server rendered, indexable) and search	`T-012`	Pages render server-side with metadata; LCP within budget; semantic search returns results online and title search works offline
`T-020`	My Garden: plants and diagnosis history	`T-004`, `T-006`	CRUD works offline and syncs on reconnect in E2E; history list uses the composite index
`T-021`	Author and review MVP content (10 crops, guides, recycled-plastic builds)	`T-012`, agronomy partner	Content targets in section 10 met; every published entry has `reviewedBy`; plastic-safety callouts reviewed
`T-022`	Golden set v1, eval runner, metrics, CI comparison	`T-014`	`pnpm eval` prints all metrics from section 11; CI comments baseline diff on pull requests
`T-023`	Calibration and threshold tuning	`T-022`	Calibration file committed; thresholds updated; wrong-confident rate meets the agreed gate

ID	Task	Depends on	Acceptance criteria
`T-024`	Privacy: consent screen, data export, account deletion	`T-006`	Emulator test shows deletion removes Auth user, Firestore subtree, and Storage prefix; export returns all user data
`T-025`	Observability and cost controls: structured logs, dashboards, alerts, budgets	`T-016`	Alerts in section 12 exist; cost per diagnosis visible on a dashboard; budget alerts fire in a test
`T-026`	Launch hardening: performance, accessibility, security review, restore drill	all above	All CI gates green on staging; restore drill documented; launch checklist signed by founder

Suggested parallel tracks

- **Platform:** `T-001`, `T-002`, `T-003`, `T-004`, `T-005`, `T-006`
- **Client:** `T-007`, `T-008`, `T-009`, `T-010`, `T-017`, `T-018`, `T-019`, `T-020`
- **AI and backend:** `T-011`, `T-013`, `T-014`, `T-015`, `T-016`, `T-022`, `T-023`
- **Content:** `T-012`, `T-021`
- **Trust and operations:** `T-024`, `T-025`, `T-026`

14. Assumptions, open questions, and decisions

Each row states the default the team builds against until the founder decides otherwise. Add new questions here rather than guessing silently.

ID	Question	Default assumed	Needed by
`Q-01`	Final product name and domain	Working name VerdeScan; no brand strings hard-coded outside tokens and i18n	Before `T-008` manifest is finalised
`Q-02`	Retrieval backend: Firestore vector search or Vertex AI Search	Firestore vector search for simplicity and offline-friendly data locality	`T-013`

ID	Question	Default assumed	Needed by
`Q-03`	Which Gemini model IDs for primary and fallback	Pick current stable Flash-class models from Firebase AI Logic docs at `T-014`; revisit each quarter	`T-014`
`Q-04`	Two-stage or single-stage pipeline as the MVP default	`two_stage`; compare accuracy and cost in `T-022`, switch via Remote Config	`T-022`
`Q-05`	Pilot region, crops, and agronomy partner	Ghana; the 10 crops listed in section 10; partner to be named	Phase 0
`Q-06`	Initial languages beyond English	`tw` and `fr` safety-critical content at launch; others in Phase 2	`T-021`
`Q-07`	Free-tier diagnosis limit and pricing	Remote Config `diag_free_monthly_limit` set to a placeholder; set after cost per diagnosis is measured	`T-025`
`Q-08`	Anonymous use allowed before sign-in	Yes for the first diagnosis, then prompt to sign in to save history	`T-006`
`Q-09`	Image licensing for KB reference photos	Only own, commissioned, or openly licensed images with recorded licence	`T-021`
`Q-10`	Legal review for data protection and health-of-plants disclaimer	Required before public launch; assume GDPR-style consent and Ghana Act 843 compliance	`T-026`
`Q-11`	Use of user photos to improve models	Opt-in only (`trainingConsent`), default off; no training use until a policy is published	`T-024`
`Q-12`	Native wrappers (Android TWA, iOS)	Out of scope for MVP; PWA only	Phase 3 review

Known platform constraints to re-check at implementation time

- Firebase App Check enforcement becomes mandatory for Firebase AI Logic from 2 November 2026, and some older Gemini model versions are scheduled for retirement in October 2026; confirm current model availability and requirements in the Firebase

docs before `T-014`.

- Firebase AI Logic itself is free to use; costs come from the chosen Gemini API provider and other Firebase products, and the Vertex AI provider requires the Blaze plan.
- Background Sync is not available in all browsers; the offline queue must work from foreground triggers alone.

Out of scope for this spec

Payments and subscriptions, the outbreak map, community Q&A, plant identification as a standalone feature, extension-officer dashboards, supplier finder, and custom or on-device models. They will get their own specs when the roadmap reaches those phases.

Sources

As of 20 October 2026.

- Firebase AI Logic pricing and requirements: App Check enforcement date, model retirements, provider and plan requirements
- Product context and scope: VerdeScan Product Scope and Investor Document (companion doc in this project)